File-System Internals in LUIT

File descriptors, inode lifetime, block boundaries, and the buffer cache

Laboratory practice | Non-graded | No submission or report

Purpose

The laboratory begins at the user-level file API and follows the effects of each operation through LUIT's open-file table, inodes, directories, block mapping, and buffer cache. Students will design experiments, implement user programs, and explain the observed behavior from the source code. No new kernel feature is required.

Scope and preparation

- Use a disposable scratch copy of LUIT that permits adding guest programs. Do NOT edit protected files or the Makefile in a graded-lab distribution.
- Use the guest interface in user/ulib.h. Do not include Linux/host stdio.h, unistd.h, or filesystem APIs in guest programs.
- Do not implement or modify rename, fsck, fast file systems (FFS), journaling, logging, transaction recovery, or crash replay.
- Experiments use ordinary file operations on disposable test filenames. Do not hex-edit the filesystem image or inject disk-write failures.

Learning outcomes

1. Explain the distinction among a per-process file descriptor, a system-wide struct file, and an inode.
2. Predict how dup() and fork() affect open-file offsets and reference counts.
3. Demonstrate that a file's name and its open inode have different lifetimes.
4. Diagnose pathname and access-mode failures using actual LUIT return values.
5. Trace read/write requests through the filesystem and buffer cache.
6. Analyze a file crossing the direct-to-indirect block boundary and verify exact data.
7. Distinguish filesystem cache activity from physical device I/O and test invariants without fsck.

Build and test convention

In an unprotected scratch source tree, place the supplied skeletons under user/ and register user/fs_semantics and user/fs_blocks in UPROGS, following the existing entries. Build and run using that source tree's standard commands, commonly:

```
make
make qemu CPUS=1
```

If the distributed Makefile is protected, do not alter it: use an instructor-provided practice tree that permits registering new user binaries. Do not run make clean on a source tree containing data you need to preserve in fs.img.

Experiment record

For each exercise, keep your own brief notes: test operation, predicted result, actual result, and the kernel function that explains it. These notes are for discussion and debugging only; nothing is submitted.

Exercise 1 — Reconstruct the file-I/O path

Before running programs, inspect these source files and draw two call-path diagrams: one for read() on a regular file and one for write() that extends a file.

```
user/ulib.h     kernel/sysfile.c   kernel/file.c
kernel/file.h   kernel/defs.h      kernel/fs.c
kernel/bio.c    kernel/fs.h        kernel/stat.h
hal/qemu_virt/virtio_blk.c  (block device)
```

1. Locate the system-call entry functions that implement open, read, write, close, dup, fstat, link, and unlink.
2. Identify where a process descriptor is translated into a struct file, and where that object stores its current byte offset.
3. Locate readi(), writei(), bmap(), bread(), bwrite(), and brelse(). Note the role of each.
4. Identify the functions and locks protecting (a) the open-file reference count, (b) inode contents, and (c) cached disk blocks.
5. Explain which path copies bytes between user and kernel memory, and which path ultimately requests block I/O.

Checkpoint: A valid answer distinguishes the descriptor table, open-file object, inode cache, buffer cache, and device driver. It does not treat them as one structure.

Exercise 2 — Shared and independent file offsets

Implement the dup mode in fs_semantics.c. Create a regular file containing exactly the eight bytes ABCDEFGH. Close the creator descriptor, then reopen the file for reading.

1. Open the file for reading as descriptor A. Duplicate A using dup() to obtain descriptor B.
2. Read two bytes through A, then two through B, and record each result.
3. Open the same filename independently as descriptor C. Read two bytes through C.
4. Read another two bytes through A. Record the sequence of bytes and explain each observed offset.
5. Close B. Verify that A is still valid. Close A and C and remove the test file.

Operation	Prediction	Observation
read(A,2)	_____	_____
read(B,2)	_____	_____
read(C,2)	_____	_____
read(A,2)	_____	_____
close(B); read(A,1)	_____	_____

Questions: Does dup() allocate a new inode? Does it allocate a new struct file? Why does independent open() behave differently? Which reference count changes on dup()?

Exercise 3 — Descriptor inheritance across fork

Implement the fork mode. Make the file contain ABCDEFGH and open it once in the parent before fork(). Use wait() to force a deterministic order: the child reads two bytes, exits, and only then the parent reads two bytes using the inherited descriptor.

1. Predict the child's two bytes and the parent's two bytes before executing.
2. Use wait() rather than sleep() to enforce the order.
3. Explain which objects are copied at fork and which object remains shared.
4. Repeat with the child independently reopening the path; compare the outcome.
5. Close all descriptors in both processes. Confirm parent/child exit paths complete.

Challenge: Why is comparing offsets in two unsynchronized concurrent readers an unreliable test of fork semantics?

Exercise 4 — One inode, multiple names, and deletion

Implement the links mode using two disposable names, e.g., fs_life_a and fs_life_b. Begin by creating fs_life_a and writing a known byte sequence.

1. Use fstat() to record the inode number, nlink, and size of the original file.
2. Create a second name for the file using link(). Open the second name and compare inode numbers and link counts.
3. Keep one descriptor open. Unlink the first name, then unlink the second name.
4. Check that open() by either deleted name fails. Use fstat() and read() on the descriptor that remains open.
5. Close the final descriptor. Explain when the inode and its blocks can become reclaimable.

State	Accessible names	nlink	Open descriptor usable?
After create	_____	_____	_____
After link	_____	_____	_____
After first unlink	_____	_____	_____
After final unlink	_____	_____	_____
After final close	_____	_____	_____

Questions: What does nlink count? What is different about the in-memory inode refcount? Is unlink() equivalent to close()? Does unlinking the final pathname necessarily invalidate an already-open descriptor?

Exercise 5 — Directories and pathname resolution

Implement the paths mode. Create a nested directory hierarchy such as fs_tree/inner and a file within it. Use relative and absolute paths, chdir(), ".", and ".." to access the same inode through different strings.

1. Create the hierarchy and a small file, then inspect its inode using fstat().
2. Change the working directory to the inner directory and open the file with a relative pathname.
3. Open it again through an equivalent path containing "." or "..". Compare inode numbers.
4. Attempt to unlink a nonempty directory and observe the return value.
5. Remove contents before removing the directories. Avoid assuming a host Linux command exists in LUIT.

Trace namei(), nameiparent(), and dirlookup() in kernel/fs.c and the directory-related system calls in kernel/sysfile.c. Explain why a pathname is not itself an inode.

Exercise 6 — Cross the direct-to-indirect allocation boundary

Implement the pattern exercise in fs_blocks.c. The starter imports BSIZE and NDIRECT from kernel/fs.h, using the relative path shown below after fs_blocks.c is copied to user/. Include ulib.h FIRST because fs.h refers to LUIT integer typedefs. Calculate every length from these constants rather than copying values by hand.

```
#include "ulib.h"
#include "../kernel/fs.h"
```

Use the following file lengths (bytes), calculating their concrete values from BSIZE and NDIRECT:

Case	Length	What this tests
P	NDIRECT × BSIZE − 1	Last byte before the direct capacity boundary
Q	NDIRECT × BSIZE	Exactly fills direct-addressed capacity
R	NDIRECT × BSIZE + 1	First byte requiring the next data block
S	(NDIRECT + 1) × BSIZE + 17	Crosses an additional data-block boundary

1. Write using multiple calls with uneven chunk lengths (for example 37, 513, and 1024 bytes). Do not assume one write corresponds to exactly one disk block.
2. After the write, use fstat() and verify the logical file size is exact.
3. Close and reopen the file instead of seeking: this LUIT interface does not expose lseek().
4. Read using chunk lengths 1, 37, 512, and 1024 bytes. Validate returned byte counts and every byte against the deterministic pattern.
5. Explain the additional metadata needed when the file grows beyond the direct-addressed blocks. Distinguish file bytes from pointer-block overhead.
6. Clean up each test file only after validating it.

Important: A partial final read at EOF is not a failure. Make sure the test accepts a positive byte count smaller than requested when fewer bytes remain.

Exercise 7 — Trace the buffer cache and actual block transfers

Inspect kernel/bio.c and the block-device function reached by bread() and bwrite(). Use GDB or temporary diagnostics in a scratch kernel. Do not modify the buffer replacement policy or locking logic.

1. Locate the lookup of a matching (device, block-number) buffer and the path that allocates/recycles a buffer.
2. Explain the difference between bread() calls and actual device reads. Identify the condition that triggers a device read on a cached buffer miss.
3. Observe a small file being opened and read twice without changing it. Compare logical read calls, buffer lookups, and device reads. Identify background metadata accesses.
4. Compare reading an existing file and writing to it. Locate when bwrite() reaches the block driver.
5. Find the role of the buffer lock versus the cache-identity spinlock. Explain why a driver operation that may sleep must not run while holding a spinlock.

Do not claim a fixed numerical cache-hit ratio: boot activity, directory/inode lookups, and cache occupancy can change the count. Explain trends and exact observed events instead.

Exercise 8 — Error handling and boundary conditions

Implement errors mode. Each experiment must report the returned value and explain why LUIT accepts or rejects the request.

Test	Observe and explain
Read from descriptor opened O_WRONLY	Permission check
Write to descriptor opened O_RDONLY	Permission check
Open nonexistent path without O_CREATE	Name lookup failure
Read from invalid descriptor	Descriptor-table check
Read at end-of-file	Return 0 versus error -1
Attempt hard link to a directory	Directory-link restriction
Attempt unlink of nonempty directory	Directory-invariant protection

Extension: Check what happens when a file is truncated using O_TRUNC, then closed and reopened. Which metadata fields and mappings should change? Inspect the current implementation instead of guessing.

Exercise 9 — Integrated consistency experiment

Build one user-space diagnostic command that creates disposable files and checks the following four properties using only documented system calls. Print one PASS/FAIL line per property:

1. A duplicate descriptor shares the same current file offset, while a separate open has an independent offset.
2. A child inherits an open-file reference after fork, and wait allows deterministic testing of the shared offset.
3. Two hard-link names refer to the same inode, and removing both names does not immediately invalidate a retained open descriptor.
4. Data written across the direct-to-indirect boundary can be read back byte-for-byte, including the last partial block.

Use unique disposable filenames. Fail explicitly on unexpected syscall results and clean up test files. This is a behavior checker, not an on-disk fsck utility.

Discussion prompts

1. Why does a file descriptor not identify a permanent location on disk?
2. Why can the inode number remain the same while file descriptors differ?
3. Which kernel structure determines a regular file's current read offset?
4. Why does deleting a filename not necessarily release the file's data immediately?
5. What does the extra indirect block store, and why is it not counted in file size?
6. Why can two logical reads produce different numbers of physical disk reads?
7. Why should no experiment assume that QEMU virtio-blk has HDD seek/rotation latency?
8. Which invariants did your experiments test, and which would require crash-recovery theory not yet covered?

Completion checklist

- All guest programs include user/ulib.h and compile with the LUIT toolchain.
- No protected graded-lab source or Makefile was changed.
- Descriptor sharing, inode lifetime, and directory observations have source-level explanations.
- Boundary data is read back and checked, not merely written.
- Buffer-cache observations distinguish cache access from device transfers.
- Every experiment leaves no unintended test files behind.

End of laboratory practice.